Table of Contents

Course Objective.....2
Course Content.....2
from clause:.....3
select clause:.....5
where clause: Parameter Binding.....7
Unique Result.....9
Pagination.....10
Joins.....12
Aggregate function.....14
Named Queries:.....16

Course Objective

Learn writing queries in Hibernate using:

- ⇧ SQL
- ⇧ Criteria API
- ⇧ HQL
- ⇧ from clause
- ⇧ select clause
- ⇧ where clause with parameters
- ⇧ unique results
- ⇧ pagination
- ⇧ joins
- ⇧ aggregate function
- ⇧ named queries

Course Content

Creating and running queries

There are 3 ways to write queries in Hibernate:

1. Hibernate Query Language (HQL)

```
session.createQuery("select b from Book b where b.title like '%hibernate%'");
```

2. Criteria API for query by Criteria (QBC)

```
session.createCriteria(Book.class).addRestrictions.like("title","%hibernate%");
```

3. Direct SQL with or without automatic mapping of resultsets to objects:

```
session.createSQLQuery("select {b.*} from Book {b} where title like '%hibernate%'").addEntity("b", Book.class);
```

Steps in Querying

1. Create a query
2. Bind run-time arguments to query parameters
3. Execute the prepared query against the database and retrieval of data.

HQL:

Hibernate's powerful query facilities allows you to write everything you write in SQL, but in object oriented terms – using classes and properties of classes.

from clause:

Below XML mapping is defined in Hibernate for Books table.

```
public class Book
{
        private int bookId;
        private String title;
        private String author;
        ...
        ...
}
```

VILP_TBL_BOOKS			
ID	**TITLE**	**CATEGORY**	**AUTHOR**
1	2 States	Fiction	Chetan Bhagat
2	The Power	Rhonda Byrne	Motivation
3	Data Structure	Technical	John Daniel
4	Revolution 2020	Fiction	Chetan Bhagat

```
<hibernate-mapping>

        <class name="vilp.example.hibernate.hql.Book" table="VILP_TBL_BOOKS">

                <id column="id" name="bookId" type="java.lang.Integer">
                ...
                </id>

                <property name="title" column="title" type="java.lang.String" />
                ...
        </class>

</hibernate-mapping>
```

Hibernate expresses everything in object oriented terms – using classes and properties of classes. It uses class name rather than table name and it uses properties of classes rather than column name.

For example, the below query fetches all persistent objects of Book – all rows from Book table.

Try it yourself:

```
String sql = "from Book";
Query query = session.createQuery(sql);

Iterator<Book> iterator = query.list().iterator();
while(iterator.hasNext())
{
        Book b = iterator.next();
}
```

Sample Output:

Hibernate will generate the below sql query representing the above HQL:

```
select
   book0_.id as id0_,
   book0_.title as title0_,
   book0_.author as author0_,
   book0_.category as category0_
from
   VILP_TBL_BOOKS book0_
```

The query select from ***VILP_TBL_BOOKS*** table, as it is mapped with the class ***vilp.example.hibernate.hql.Book.*** Hence, the query displays all the columns & rows from the table ***VILP_TBL_BOOKS.***

select clause:

```
public class Book
{
        private int bookId;
        private String title;
        private String author;
        ...
        ...
}
```

VILP_TBL_BOOKS			
ID	**TITLE**	**CATEGORY**	**AUTHOR**
1	2 States	Fiction	Chetan Bhagat
2	The Power	Rhonda Byrne	Motivation
3	Data Structure	Technical	John Daniel
4	Revolution 2020	Fiction	Chetan Bhagat

```
<hibernate-mapping>

        <class name="vilp.example.hibernate.hql.Book" table="VILP_TBL_BOOKS">

                <id column="id" name="bookId" type="java.lang.Integer">
                ...
                </id>

                <property name="title" column="title" type="java.lang.String" />
                ...
        </class>

</hibernate-mapping>
```

Hibernate expresses everything in object oriented terms – using classes and properties of classes. It uses class name rather than table name and it uses properties of classes rather than column name.

For example, the below query fetches title and author for all persistent objects of Book – all rows from Book table.

Try it yourself:

```
String sql = "select title, author from Book"

Query query = session.createQuery(sql);

Iterator<Object[]> iterator = query.list().iterator();

while(iterator.hasNext())
{
        Object[] row = iterator.next();
}
```

Sample Output:

Hibernate will generate the below sql query representing the above HQL:

```
select
   book0_.title as col_0_0_,
   book0_.author as col_1_0_
from
   VILP_TBL_BOOKS book0_
```

The query select from ***VILP_TBL_BOOKS*** table, as it is mapped with the class ***vilp.example.hibernate.hql.Book.*** Hence, the query displays the ***title*** and ***author*** columns for all rows from the table ***VILP_TBL_BOOKS.***

where clause: Parameter Binding

When you use parameters, the database is able to efficiently cache pre-compiled prepared statement, hence improving performance significantly.

There are 2 approaches to parameter binding

1. Positional parameters
2. Named parameters

Named Parameters :

Try it yourself

```
String sql = "from Book where title = :search";
String titleCriteria = "Revolution 2020";

Query query = session.createQuery(sql);
query.setString("search", titleCriteria);

Iterator<Book> iterator = query.list().iterator();
```

The colon followed by a parameter name indicates a named parameter. Then, bind a value to the named parameter. The corresponding setter method is called for Because ***titleCriteria*** is a user supplied string variable, you call the ***setString()*** method of the Query interface to bind it to the named parameter ***:search***. Named parameter queries are self documenting.

Sample Output:

Hibernate will generate the below sql query representing the above HQL:

```
select
   book0_.id as id0_,
   book0_.title as title0_,
   book0_.author as author0_,
   book0_.category as category0_
from
   VILP_TBL_BOOKS book0_
where
   book0_.title=?
```

The query select from ***VILP_TBL_BOOKS*** table, as it is mapped with the class ***vilp.example.hibernate.hql.Book.*** Hibernate also puts a where clause on the attribute on the class attribute ***title.*** Hence, the query displays all rows from the table ***VILP_TBL_BOOKS*** matching the criteria on the column ***title***, as the class attribute ***title*** is mapped with column ***title.***

Positional Parameters

The usage of positional parameters in Hibernate is similar to that in PreparedStatment. Hibernate starts counting of positional parameters from 0. Every change in the position of the bind parameters requires a change to the parameter binding code. This leads to fragile and maintenance-intensive code. Hibernate recommends to avoid positional parameters.

Try it yourself

```
String sql = "from Book where title=?";
String titleCriteria = "Revolution 2020";

Query query = session.createQuery(sql);
query.setString(0, titleCriteria);

Iterator<Book> iterator = query.list().iterator();
```

Sample Output:

Hibernate will generate the below sql query representing the above HQL:

```
select
   book0_.id as id0_,
   book0_.title as title0_,
   book0_.author as author0_,
   book0_.category as category0_
from
   VILP_TBL_BOOKS book0_
where
   book0_.title=?
```

The query select from ***VILP_TBL_BOOKS*** table, as it is mapped with the class ***vilp.example.hibernate.hql.Book.*** Hibernate also puts a where clause on the attribute on the class attribute ***title.*** Hence, the query displays all rows from the table ***VILP_TBL_BOOKS*** matching the criteria on the column ***title***, as the class attribute ***title*** is mapped with column ***title.***

Unique Result

With some queries you know that the result is only a single instance. You can read it from the result list by index, list.get(0) or you can limit the number of returned rows with ***setMaxResult(1)***. Then you may execute the query with the ***uniqueResult()*** method, because you know only one object will be returned.

Try it yourself

```
String sql = "from Book where ID =  ?";
String bookId = "2";

Query query = session.createQuery(sql);
query.setString(0, bookId);
query.setMaxResults(1);

Book book = (Book)query.uniqueResult();
```

Pagination

Users may see the result of their search request (for example, an item search on flipkart) as multiple pages. A page shows a limited subset (say, 10 items) at a time, and users can navigate to the next and previous pages manually. In Hibernate, Query interface supports this pagination of the query result.

Try it yourself

```
String sql = "from Book";

Query query = session.createQuery(sql);
query.setMaxResults(10);

Iterator<Book> iterator = query.list().iterator();
```

In the above query, the call to ***setMaxResults(10)*** limits the query resultset to the first 10 objects (row) returned by the database.

Sample Output:

Hibernate will generate the below sql query representing the above HQL:

```
select
   *
from
   ( select
      book0_.id as id0_,
      book0_.title as title0_,
      book0_.author as author0_,
      book0_.category as category0_
   from
      VILP_TBL_BOOKS book0_ )
where
   rownum <= ?
```

Try it yourself

```
String sql = "from Book";

Query query = session.createQuery(sql);
query.setFirstResult(20);
query.setMaxResults(10);

Iterator<Book> iterator = query.list().iterator();
```

In the above query, the requested page starts in the middle of the resultset. Starting from the ***twentieth*** object, you retrieve the next ***ten*** objects.

Sample Output:

Hibernate will generate the below sql query representing the above HQL:

```
select
   *
from
   ( select
      row_.*,
      rownum rownum_
   from
      ( select
         book0_.id as id0_,
         book0_.title as title0_,
         book0_.author as author0_,
         book0_.category as category0_
      from
         VILP_TBL_BOOKS book0_ ) row_
   where
      rownum <= ?
   )
where
   rownum_ > ?
```

Joins

In HQL, you don't usually specify a join condition explicitly. Rather, you specify the name of a mapped Java class association. Because you have mapped foreign key relationships of your database schema in Hibernate, you can use the names of these mapped associations in the query language.

For example, The ***BookTransactionDto*** class has an association named ***book*** with the ***BookDto*** class. If you use this association (***book***) in a query, Hibernate has enough information in the XML mapping to then deduce the table join expression.

Try it yourself

```
public class BookTransactionDto
{
        private int transactionId;
        private Date issueDate;
        private Date dueDate;

        private Customer customerDto;
        private Book bookDto;

        ...
        ...
}
```

```
public class Book
{
        private int bookId;
        private String title;
        private String author;
        ...
        ...
}

public class Customer
{
        private int id;
        private String name;
        private String email;
        private String address;
        ...
        ...
}
```

VILP_TBL_BOOKS			
ID	**TITLE**	**CATEGORY**	**AUTHOR**
1	2 States	Fiction	Chetan Bhagat
2	The Power	Rhonda Byrne	Motivation
3	Data Structure	Technical	John Daniel
4	Revolution 2020	Fiction	Chetan Bhagat

VILP_CUSTOMERS			
CUSTOMER ID	**NAME**	**EMAIL**	**ADDRESS**
1	AJAY	AJAY@VILP.COM	STUDIO, INFOCITY, GANDHINAGAR
2	ROHIT	ROHIT@VILP.COM	MADHURAM, GANDHINAGAR
3	GOPAL	GOPAL@VILP.COM	MADHURAM, GANDHINAGAR
4	AMIT	AMIT@VILP.COM	STUDIO, INFOCITY, GANDHINAGAR

VILP_BOOK_TRANSACTION				
TRANSACTION ID	**CUSTOMER ID**	**BOOK ID**	**ISSUED DATE**	**DUE DATE**
1	1	4	1. May. 2014	15. May. 2014
2	2	2	1. May. 2014	15. May. 2014
3	3	2	3. May. 2014	18. May. 2014
4	2	1	7. May. 2014	21. May. 2014
5	1	2	11. May. 2014	26. May. 2014
6	2	3	15. May. 2014	30. May. 2014
7	3	3	15. May. 2014	30. May. 2014

Try it yourself

```
String sql = "SELECT      C.name AS CUSTOMER, B.title AS BOOKTITLE, B.author AS BOOKAUTHOR, " +
          "               BT.issueDate, BT.dueDate " +
          "FROM           BookTransactionDto BT " +
          "INNER JOIN     BT.customerDto C " +
          "INNER JOIN     BT.bookDto B " +
          "WHERE          C.id = ?";

Query query = session.createQuery(sql);
query.setString(0, "2");

Iterator<Object[]> iterator = query.list().iterator();
while(iterator.hasNext())
{
          Object[] row = iterator.next();
}
```

Sample Output:

Hibernate will generate the below sql query representing the above HQL:

```
select
   customer1_.NAME as col_0_0_,
   book2_.title as col_1_0_,
   book2_.author as col_2_0_,
   booktransa0_.issueDate as col_3_0_,
   booktransa0_.dueDate as col_4_0_
from
   VILP_TBL_BOOKTRANSACTION booktransa0_
inner join
   VILP_TBL_CUSTOMER customer1_
      on booktransa0_.customerId=customer1_.id
inner join
   VILP_TBL_BOOKS book2_
      on booktransa0_.bookId=book2_.id
where
   customer1_.id=?
```

Aggregate function

Aggregate functions in HQL can be used similar to that in SQL. Just like in SQL, any property or alias that appears in HQL outside of an aggregate function in the SELECT clause must also appear in the GOUP BY clause.

For example, the below query counts the number of ***transaction ids associated*** with each ***book id***.

Try it yourself

```
String sql = "SELECT      bookDto.bookId AS BOOKID, COUNT(transactionId) AS ISSUEDCOUNT " +
                  "FROM           BookTransactionDto BT " +
                  "GROUP BY       BT.bookDto ";

Query query = session.createQuery(sql);
Iterator<Object[]> iterator = query.list().iterator();

while(iterator.hasNext())
{
        Object[] row = iterator.next();
}
```

Sample Output:

Hibernate will generate the below sql query representing the above HQL:

```
select
   booktransa0_.bookId as col_0_0_,
   count(booktransa0_.transactionId) as col_1_0_
from
   VILP_TBL_BOOKTRANSACTION booktransa0_
group by
   booktransa0_.bookId
```

VILP_BOOK_TRANSACTION				
TRANSACTION ID	**CUSTOMER ID**	**BOOK ID**	**ISSUED DATE**	**DUE DATE**
1	1	4	1. May. 2014	15. May. 2014
2	2	2	1. May. 2014	15. May. 2014
3	3	2	3. May. 2014	18. May. 2014
4	2	1	7. May. 2014	21. May. 2014
5	1	2	11. May. 2014	26. May. 2014
6	2	3	15. May. 2014	30. May. 2014
7	3	3	15. May. 2014	30. May. 2014

Considering the above sample data, the HQL will give result as follows:

BOOKID	ISSUEDCOUNT
1	1
2	3
21	2
22	1

Named Queries:

We don't like to see HQL (String literals) scattered all over the Java code, unless really necessary. Hibernate lets you externalize query strings to the mapping metadata. This technique of externalizing query is called ***named queries***. This lets you store all queries related to a particular persistent scope in an XML mapping file. The name of the query is used to call it from application code.

Try it yourself:

```
<hibernate-mapping>

<class name="vilp.example.hibernate.hql.Customer" table="VILP_TBL_CUSTOMER">

        <id ... >
        ...
        </id>

        <property ... />

</class>

<query name="viewAllCustomerQuery">from Customer order by name</query>

</hibernate-mapping>
```

```
Query query = session.getNamedQuery("viewAllCustomerQuery");

Iterator<Customer> iterator = query.list().iterator();

while(iterator.hasNext())
{
        Customer customer = iterator.next();
}
```

In the example above, the ***query*** tag's attribute ***name*** defines the name of the query. The query is written as a text data.

The ***getNamedQuery()*** method obtains a Query instance for a named query.

Named queries are global – that is, the name of a query is considered to be a unique identifier for a particular ***Session Factory***.

Sample Output:

Hibernate will generate the below sql query representing the above HQL:

```
select
   customer0_.id as id1_,
   customer0_.NAME as NAME1_,
   customer0_.email as email1_,
   customer0_.address as address1_
from
   VILP_TBL_CUSTOMER customer0_
order by
   customer0_.NAME
```